블록 저장소 구조

개요

CometBFT의 블록 저장소(BlockStore)는 블록체인의 모든 블록 데이터를 효율적으로 저장하고 검색하는 핵심 컴포넌트입니다. 블록 저장소는 블록 메타데이터, 블록 파트, 커밋 정보를 체계적으로 관리하며, 빠른 조회와 안전한 데이터 무결성을 보장합니다.

핵심 개념

저장되는 데이터 유형

CometBFT 블록 저장소는 다음 세 가지 주요 데이터 유형을 관리합니다:

1. **BlockMeta**: 각 블록의 메타데이터 정보
2. **Block Parts**: 블록을 여러 부분으로 나눈 데이터 조각들
3. **Commit**: 각 블록에 대한 커밋 정보 (precommit votes)

연속성 보장

블록 저장소는 `base` 와 `height` 사이의 모든 블록이 연속적으로 존재함을 보장합니다:

- **Base**: 저장소에서 사용 가능한 첫 번째 블록 높이
- **Height**: 저장소에서 사용 가능한 마지막 블록 높이
- **Size**: `height - base + 1` 로 계산되는 총 블록 수

아키텍처

BlockStore 구조체

```go
type BlockStore struct {
db dbm.DB // 데이터베이스 인터페이스
metrics *Metrics // 성능 메트릭
// 동시성 제어
mtx cmtsync.RWMutex
base  int64  // 첫 번째 블록 높이
height  int64  // 마지막 블록 높이
dbKeyLayout BlockKeyLayout // 키 레이아웃 버전
// 성능 최적화
seenCommitCache *lru.Cache[int64, *types.Commit]
blockCommitCache *lru.Cache[int64, *types.Commit]
blockExtendedCommitCache *lru.Cache[int64, *types.ExtendedCommit]
blockPartCache *lru.Cache[blockPartIndex, *types.Part]
// 압축 설정
blocksDeleted  int64
compact  bool
compactionInterval  int64
}
```

데이터베이스 키 레이아웃

CometBFT는 두 가지 키 레이아웃 버전을 지원합니다:

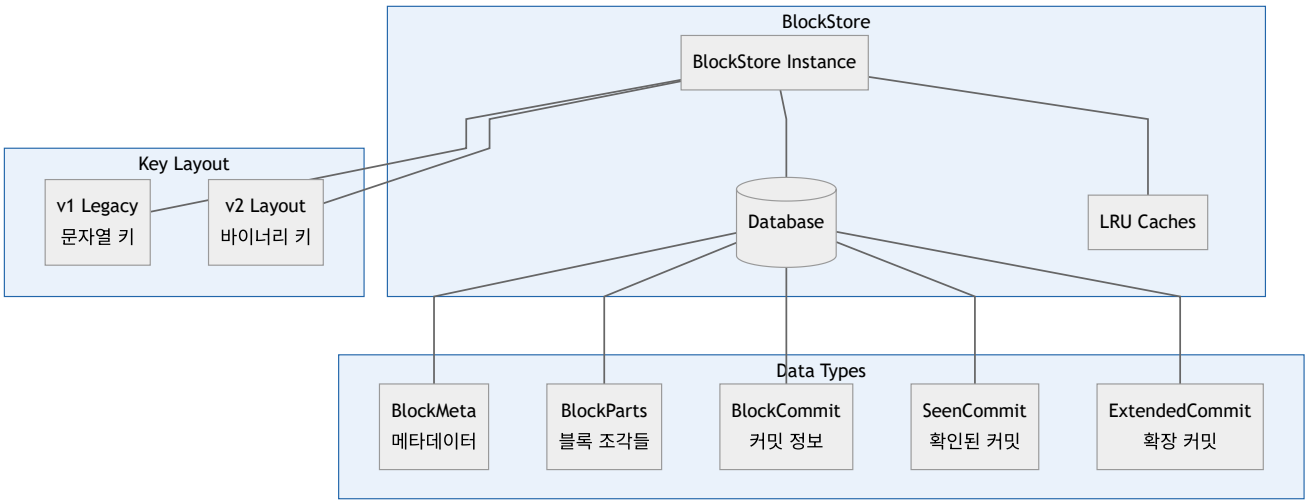
v1 Legacy Layout (기본)

```go
type v1LegacyLayout struct{}

// 키 형식:
// - 블록 메타: "H:<height>"
// - 블록 파트: "P:<height>:<partIndex>"
// - 블록 커밋: "C:<height>"
// - 확인된 커밋: "SC:<height>"
// - 확장 커밋: "EC:<height>"
// - 블록 해시: "BH:hex(<hash>)"
```

v2 Layout (최적화된 버전)

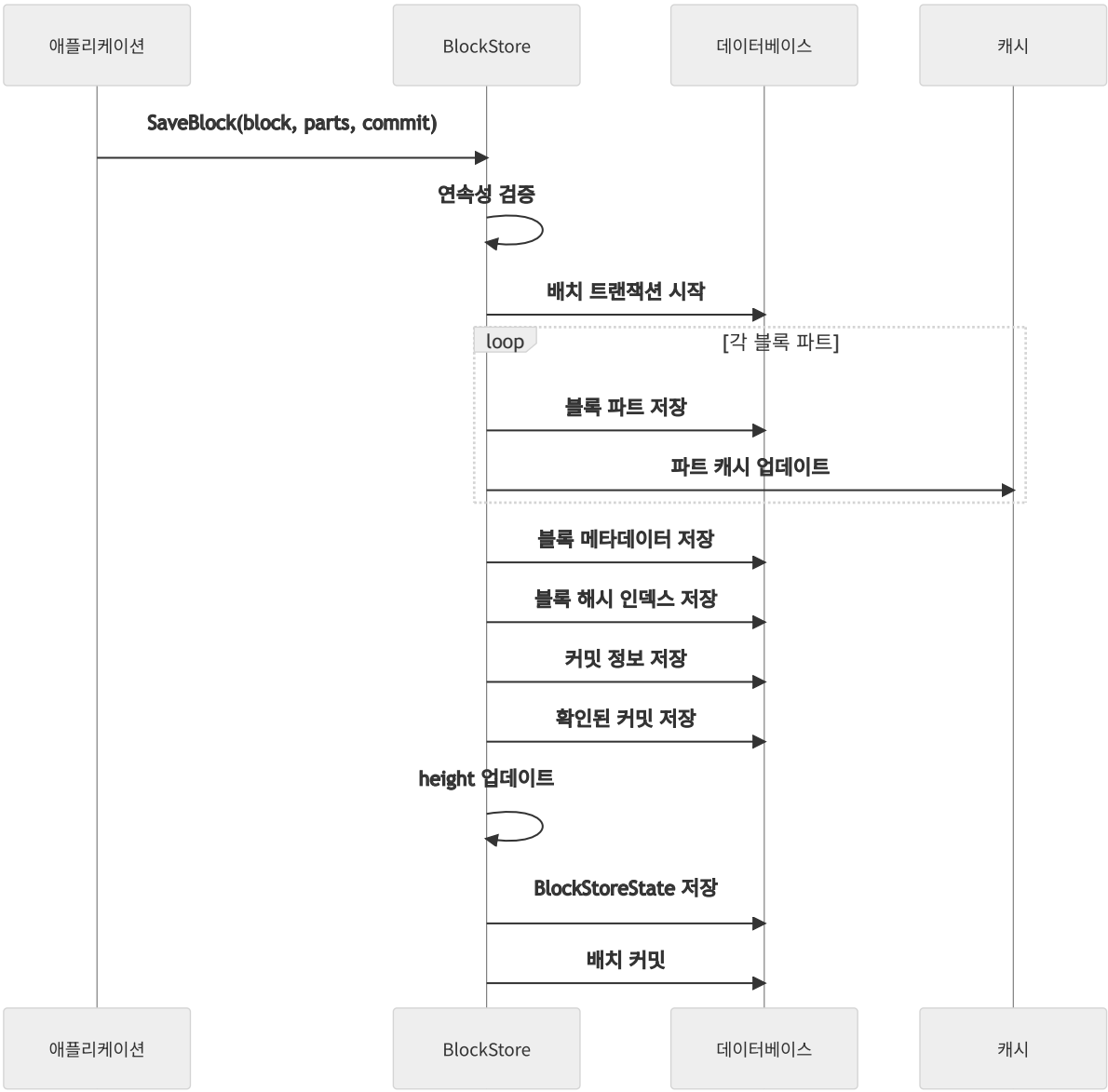
```go
type v2Layout struct{}

// orderedcode를 사용한 바이너리 키 인코딩
// - 더 효율적인 저장 공간 사용
// - 향상된 성능
// - 정렬된 키 순서 보장
```

저장 구조 다이어그램

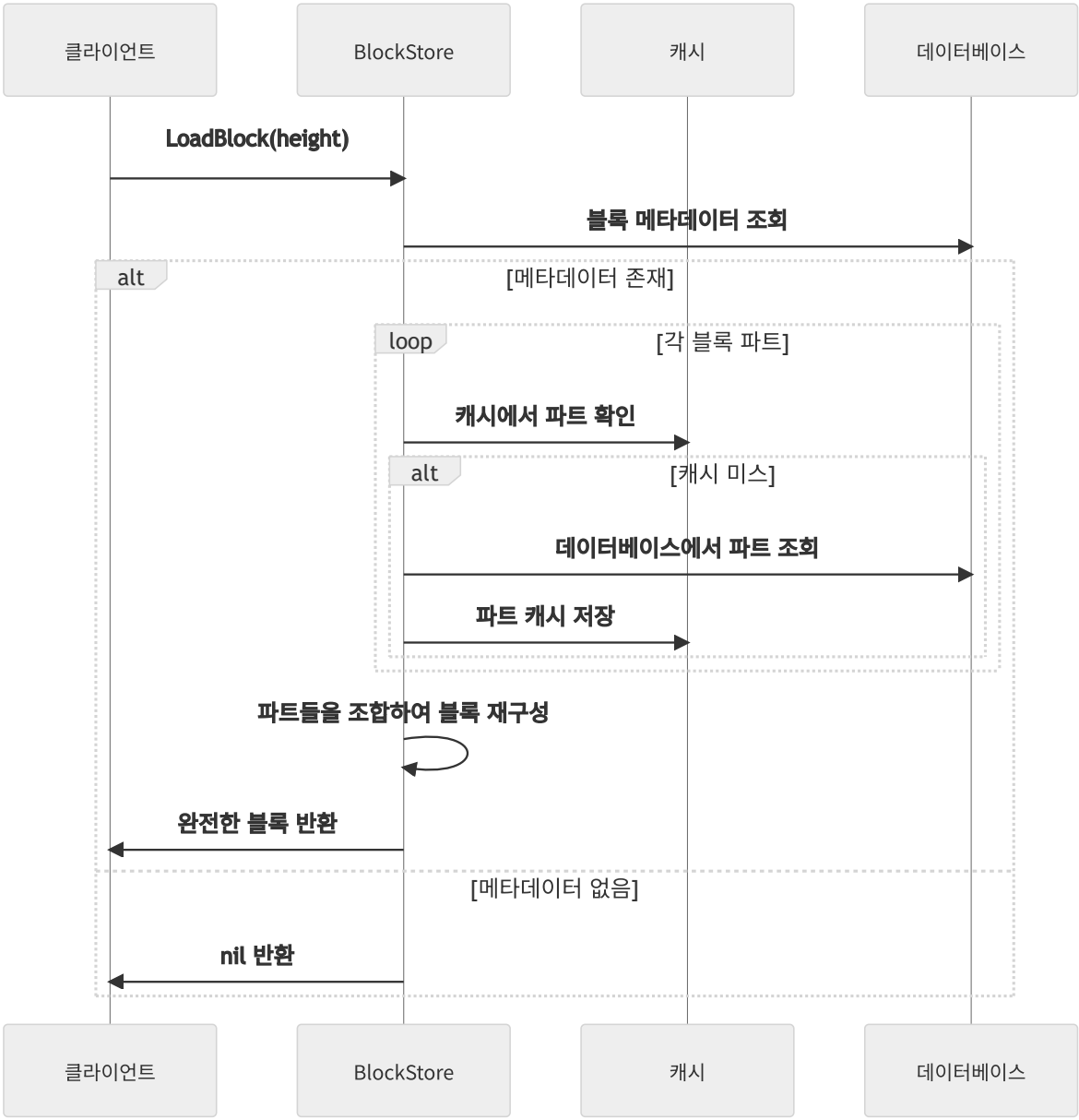

BlockStore
BlockStore Instance
Key Layout
v1 Legacy
문자열 키
v2 Layout
바이너리 키
Database
LRU Caches
Data Types
BlockMeta
메타데이터
BlockParts
블록 조각들
BlockCommit
커밋 정보
SeenCommit
확인된 커밋
ExtendedCommit
확장 커밋

동작 과정

블록 저장 과정


애플리케이션
BlockStore
데이터베이스
캐시
SaveBlock(block, parts, commit)
연속성 검증
배치 트랜잭션 시작
loop
[각 블록 파트]
블록 파트 저장
파트 캐시 업데이트
블록 메타데이터 저장
블록 해시 인덱스 저장
커밋 정보 저장
확인된 커밋 저장
height 업데이트
BlockStoreState 저장
배치 커밋
애플리케이션
BlockStore
데이터베이스
캐시

블록 조회 과정



코드 분석

블록 저장 구현

```
func (bs *BlockStore) SaveBlock(block *types.Block, blockParts *types.PartSet, s
```

```
if  block == nil {
panic("BlockStore can only save a non-nil block")
}

batch := bs.db.NewBatch()
defer  batch.Close()

if  err := bs.saveBlockToBatch(block, blockParts, seenCommit, batch);  err != ni
panic(err)
}

bs.mtx.Lock()
defer  bs.mtx.Unlock()
bs.height = block.Height
if  bs.base == 0 {
bs.base = block.Height
}

// BlockStoreState 저장 및 데이터베이스 플러시
err := bs.saveStateAndWriteDB(batch, "failed to save block")
if  err != nil {
panic(err)
}
```

```
}
```

배치 저장 최적화

```
func (bs *BlockStore) saveBlockToBatch(
block *types.Block,
blockParts *types.PartSet,
seenCommit *types.Commit,
batch dbm.Batch,
) error {
height := block.Height
hash := block.Hash()

// 연속성 검증
if  g, w := height, bs.Height()+1;  bs.Base() > 0 && g != w {
return  fmt.Errorf("BlockStore can only save contiguous blocks. Wanted %v, got %
}

// 블록 파트 저장 (작은 블록은 배치로, 큰 블록은 개별로)
saveBlockPartsToBatch := blockParts.Count() <= maxBlockPartsToBatch

for  i := 0;  i < int(blockParts.Total());  i++ {
```

```
part := blockParts.GetPart(i)

bs.saveBlockPart(height, i, part, batch, saveBlockPartsToBatch)

bs.blockPartCache.Add(blockPartIndex{height, i}, part)

}



// 블록 메타데이터 저장

blockMeta := types.NewBlockMeta(block, blockParts)

pbm := blockMeta.ToProto()

metaBytes := mustEncode(pbm)

if  err := batch.Set(bs.dbKeyLayout.CalcBlockMetaKey(height), metaBytes);  err !

return  err

}

// 해시 인덱스 저장

if  err := batch.Set(bs.dbKeyLayout.CalcBlockHashKey(hash), []byte(strconv.Forma

return  err

}



// 커밋 정보 저장

pbc := block.LastCommit.ToProto()

blockCommitBytes := mustEncode(pbc)

if  err := batch.Set(bs.dbKeyLayout.CalcBlockCommitKey(height-1), blockCommitByt

return  err

}
```

```go
// 확인된 커밋 저장
pbsc := seenCommit.ToProto()
seenCommitBytes := mustEncode(pbsc)
if err := batch.Set(bs.dbKeyLayout.CalcSeenCommitKey(height), seenCommitBytes);
return err
}

return nil
}
```

블록 조회 구현

```go
func (bs *BlockStore) LoadBlock(height int64) (*types.Block, *types.BlockMeta)
blockMeta := bs.LoadBlockMeta(height)
if blockMeta == nil {
return nil, nil
}
pbb := new(cmtproto.Block)
buf := []byte{}
// 모든 블록 파트를 순서대로 조합
for i := 0; i < int(blockMeta.BlockID.PartSetHeader.Total); i++ {
part := bs.LoadBlockPart(height, i)
```

```go
if part == nil {
return nil, nil // 파트가 누락된 경우 전체 블록을 누락으로 처리
}
buf = append(buf, part.Bytes...)
}
// 프로토콜 버퍼 역직렬화
err := proto.Unmarshal(buf, pbb)
if err != nil {
panic(fmt.Sprintf("Error reading block: %v", err))
}
block, err := types.BlockFromProto(pbb)
if err != nil {
panic(cmterrors.ErrMsgFromProto{MessageName: "Block", Err: err})
}

return block, blockMeta
}
```

캐시 활용 최적화

```go
func (bs *BlockStore) LoadBlockPart(height int64, index int) *types.Part {
// 캐시에서 먼저 확인
part, ok := bs.blockPartCache.Get(blockPartIndex{height, index})
```

```go
if ok {
return part
}
// 캐시 미스 시 데이터베이스에서 조회
pbpart := new(cmtproto.Part)
bz, err := bs.db.Get(bs.dbKeyLayout.CalcBlockPartKey(height, index))
if err != nil {
panic(err)
}


if len(bz) == 0 {
return nil
}
err = proto.Unmarshal(bz, pbpart)
if err != nil {
panic(fmt.Errorf("unmarshal to cmtproto.Part failed: %w", err))
}
part, err = types.PartFromProto(pbpart)
if err != nil {
panic(fmt.Sprintf("Error reading block part: %v", err))
}
// 캐시에 저장
bs.blockPartCache.Add(blockPartIndex{height, index}, part)
```

```go
return part
}
```

설정 및 튜닝

캐시 크기 설정

```go
func (bs *BlockStore) addCaches() {
var err error
// 각 캐시의 크기 설정
bs.blockCommitCache, err = lru.New[int64, *types.Commit](100)
bs.blockExtendedCommitCache, err = lru.New[int64, *types.ExtendedCommit](100)
bs.seenCommitCache, err = lru.New[int64, *types.Commit](100)
bs.blockPartCache, err = lru.New[blockPartIndex, *types.Part](500)
}
```

압축 설정

```go
// 압축 활성화 옵션
func WithCompaction(compact bool, compactionInterval int64) BlockStoreOption
return func(bs *BlockStore) {
bs.compact = compact
bs.compactionInterval = compactionInterval
```

```
}
}

// 사용 예시
blockStore := NewBlockStore(db,
WithCompaction(true, 10000), // 10,000 블록마다 압축
WithMetrics(metrics),
)
```

배치 크기 최적화

```
// 배치로 저장할 최대 블록 파트 수
const maxBlockPartsToBatch = 10

// 블록 크기에 따른 저장 전략
func (bs *BlockStore) saveBlockPart(height int64, index int, part *types.Part,
pbp, err := part.ToProto()
if err != nil {
panic(cmterrors.ErrMsgToProto{MessageName: "Part", Err: err})
}

partBytes := mustEncode(pbp)
```

```
if saveBlockPartsToBatch {
// 작은 블록: 배치에 추가
err = batch.Set(bs.dbKeyLayout.CalcBlockPartKey(height, index), partBytes)
} else {
// 큰 블록: 즉시 저장
err = bs.db.Set(bs.dbKeyLayout.CalcBlockPartKey(height, index), partBytes)
}
if err != nil {
panic(err)
}
}
```

블록 정리 및 관리

블록 정리 (Pruning)

```
func (bs *BlockStore) PruneBlocks(height int64, state sm.State) (uint64, int64,
if height <= 0 {
return 0, -1, ErrNegativeHeight
}
bs.mtx.RLock()
if height > bs.height {
bs.mtx.RUnlock()
```

```
return 0, -1, ErrExceedLatestHeight{Height: bs.height}
}
base := bs.base
bs.mtx.RUnlock()
pruned := uint64(0)
batch := bs.db.NewBatch()
defer batch.Close()
evidencePoint := height
for h := base; h < height; h++ {
meta := bs.LoadBlockMeta(h)
if meta == nil {
continue
}

// 증거 보존 기간 확인
if evidencePoint == height && !evidence.IsEvidenceExpired(
state.LastBlockHeight, state.LastBlockTime, h, meta.Header.Time,
state.ConsensusParams.Evidence) {
evidencePoint = h
}

// 증거 기간 이후의 데이터만 삭제
if h < evidencePoint {
```

```
if err := batch.Delete(bs.dbKeyLayout.CalcBlockMetaKey(h)); err != nil {
return 0, -1, ErrDBOpt{Err: err}
}
if err := batch.Delete(bs.dbKeyLayout.CalcBlockCommitKey(h)); err != nil {
return 0, -1, ErrDBOpt{Err: err}
}
}
// 해시 인덱스와 확인된 커밋은 항상 삭제
if err := batch.Delete(bs.dbKeyLayout.CalcBlockHashKey(meta.BlockID.Hash)); er
return 0, -1, ErrDBOpt{Err: err}
}
if err := batch.Delete(bs.dbKeyLayout.CalcSeenCommitKey(h)); err != nil {
return 0, -1, ErrDBOpt{Err: err}
}
// 블록 파트들 삭제
for p := 0; p < int(meta.BlockID.PartSetHeader.Total); p++ {
if err := batch.Delete(bs.dbKeyLayout.CalcBlockPartKey(h, p)); err != nil {
return 0, -1, ErrDBOpt{Err: err}
}
}
pruned++

// 1000개마다 배치 플러시
```

```
if pruned%1000 == 0 && pruned > 0 {
err := bs.flushBatch(batch, h)
if err != nil {
return 0, -1, ErrDBOpt{Err: err}
}
batch = bs.db.NewBatch()
defer batch.Close()
}
}

return pruned, evidencePoint, nil
}
```

문제 해결

일반적인 블록 저장소 이슈

1. 블록 연속성 오류

```
Error: BlockStore can only save contiguous blocks. Wanted 101, got 103
```

해결 방법:

- 누락된 블록 높이 확인
- 블록 동기화 상태 점검
- 네트워크 연결 상태 확인

2. 블록 파트 누락

```
Error: Block part missing for height 12345, part 2
```

해결 방법:

- 블록 저장 과정에서 중단 여부 확인
- 데이터베이스 무결성 검사
- 블록 재동기화 수행

3. 데이터베이스 용량 부족

```
Error: failed to save block: disk space insufficient
```

해결 방법:

- 디스크 공간 확보
- 블록 정리(pruning) 설정 조정
- 압축 기능 활성화

성능 최적화

캐시 히트율 모니터링

```go
// 캐시 성능 메트릭 수집
func (bs *BlockStore) getCacheStats() map[string]interface{} {
return  map[string]interface{}{
"block_commit_cache_len": bs.blockCommitCache.Len(),
"block_part_cache_len": bs.blockPartCache.Len(),
```

```
"seen_commit_cache_len": bs.seenCommitCache.Len(),
"extended_commit_cache_len": bs.blockExtendedCommitCache.Len(),
}
}
```

데이터베이스 압축 모니터링

```
func (bs *BlockStore) shouldCompact() bool {
return bs.compact && bs.blocksDeleted >= bs.compactionInterval
}

func (bs *BlockStore) performCompaction() error {
if bs.shouldCompact() {
err := bs.db.Compact(nil, nil)
if err == nil {
bs.blocksDeleted = 0 // 압축 성공 시 카운터 리셋
}
return err
}
return nil
}
```

디버깅 도구

블록 저장소 상태 확인

```
# 블록 저장소 정보 조회
cometbft show-block-store

# 특정 높이 블록 정보 확인
cometbft show-block --height 12345

# 블록 저장소 무결성 검사
cometbft verify-block-store --from 1 --to 1000
```

성능 메트릭 모니터링

```
// 주요 성능 지표
type Metrics struct {
BlockStoreAccessDurationSeconds metrics.Histogram
BlockStoreCacheHitRate metrics.Gauge
BlockStoreSize metrics.Gauge
BlockStorePrunedBlocks metrics.Counter
}
```

참고 자료

- [CometBFT Store 패키지 문서]

- 데이터베이스 백엔드 비교
- 상태 구조 문서
- 머클 트리와 상태 증명
- 성능 최적화 가이드